FreezeFrame

Banks catch cross-bank money-mule chains together, in real time, inside a confidential enclave on IBM LinuxONE, without any bank seeing another bank's customers.

Contents

1. System architecture
2. How a payment flows (with measured stage latencies)
3. Security flow and threat model
4. LinuxONE deployment specification
5. What is built

Headline results (IBM AMLworld HI-Small, 1,015,669 test payments)

	Each bank alone	Together in the enclave
Minority-class F1	0.559 (mean of 3 deals, 0.535–0.580)	**0.630** (published baseline 0.632)
Laundering payments frozen automatically	344–389 of 1,797	**611** of 1,797
Legitimate payments frozen	4–11	12

Streaming: p99 decision latency under 50 ms up to 539 payments/s with the warm tier running, 2,448/s hot path alone (CPU scoring; the IBM Z AI accelerator is picked up automatically when present).

In one paragraph

Each bank verifies the enclave's attestation, then signs and seals its own payments to it. Inside, account numbers become keyed pseudonyms before any processing. IBM Research's Graph Feature Preprocessor (Snap ML) mines fan-in/out, cycle and scatter-gather patterns as payments stream in; a gradient-boosted model scored through Snap ML (IBM Z AI accelerator when present) yields a calibrated probability; a cost rule decides release, hold or freeze. Payments out of a flagged account are held at the next hop. Each decision goes back signed, only to the bank that owns the payment, with reason codes naming a pattern family, never another bank's customer.

Why pooling, not federated learning

Federated learning shares model updates, but each bank's graph features would still come from its own slice, so a chain crossing banks is never seen whole. The enclave sees the whole chain under pseudonyms and returns only per-owner decisions.

All figures come from results/HI-Small in the repository.

Three banks, one enclave, decisions only to the owner

Bank A gateway
BankClient: own payments only; 5 ms batching

Bank B gateway
Ed25519 signing key + X25519 box key

Bank C gateway
verifies enclave attestation before first send

Bus: Kafka / IBM Event Streams
`payments.in` (1 partition, time order)
`decisions.<m>` , `notices.<m>` per member

→ signed + sealed

ENCLAVE, IBM LINUXONE, SECURE EXECUTION GUEST (HYPER PROTECT)

HotConsumer
open box, verify signature vs registered key, member match, freshness, nonce replay cache, micro-batch (2 ms)

Engine._frame
validate fields, ownership (sending bank ∈ member), HMAC pseudonyms, drop raw accounts

Graph Feature Preprocessor (hot)
Snap ML GFP, one `transform` per timestamp: fan, degree, temporal cycle, length-constrained cycle, vertex stats, hub splitting

Booster + calibration
XGBoost-trained trees imported into Snap ML (`zdnn_tensors` on the Z AI accelerator, else CPU), isotonic calibration

Decide
hold if p × amount > $100 review cost, freeze if p ≥ 95%-precision threshold, watchlist: hold next hop (hubs exempt)

Explain + route
reason codes by evidence family, seal + sign to owner, freeze notice to receiving bank (its account only)

Warm tier (own process) every 2–3 s: full GFP incl. scatter-gather + full model on recent payments; escalates released-but-suspicious payments to the shared watchlist and notifies the receiving bank

Also inside: member registry (one-time enrollment tokens), pseudonym key (sealed storage), attestation document (keys + code measurement + model digests)

→ signed + sealed

decisions.<paying bank>
release / hold / freeze, probability, reason codes, model version

notices.<receiving bank>
freeze request or escalation hold naming only its own account; no sender details

/v1/metrics (all)
consortium counts, latency, security events; no transaction data

HTTP alternative
`enclave/service.py` : attestation, health, enroll, rotate, submit, notices

Code: freezeframe/enclave/{stream,engine,warm,crypto,identity,attest,service}.py, features.py, model.py, decide.py, explain.py

From a bank's gateway to a decision in milliseconds

1. **Bank gateway** batches its own payments for up to 5 ms, signs the batch (Ed25519) and seals it to the attested enclave key (X25519 sealed box), then publishes it on `payments.in`.
2. **Enclave consumer** collects a micro-batch (2 ms linger), opens each message and checks signature, member, freshness (120 s) and nonce. Failures are refused and counted; they never stop the consumer.
3. **Validation and ownership:** every field is checked; the sending bank must belong to the submitting member.
4. **Pseudonymise:** HMAC-SHA256(enclave key, bank:account), truncated to 52 bits. Raw account strings are dropped (only receiving accounts are kept, to address freeze notices).
5. **Graph features:** GFP `transform` once per timestamp group, exactly as in training (tested: streaming = batch, no look-ahead).
6. **Score:** Snap ML booster → isotonic calibration → probability.
7. **Decide:** release / hold / freeze; the watchlist holds payments out of accounts that just received flagged money.
8. **Explain:** for held or frozen payments only, reason codes = attributions summed per evidence family.
9. **Publish:** each decision is signed by the enclave and sealed to its owner; freeze notices go to the receiving bank.
10. **Warm tier** re-scores the same payments with all patterns a few seconds later and escalates misses.

Measured stage latency, 100× real time (HI-Small, laptop x86_64)

Stage	p50 ms	p99 ms
Open sealed messages	0.38	1.88
Prepare + pseudonymise	0.11	0.33
Graph features (GFP)	0.46	3.16
Score (Snap ML, CPU)	0.13	0.45
Decide	0.03	0.09
Publish (seal to owner)	0.51	2.01
End to end, scheduled arrival → published decision, incl. 5 ms bank batching	**11.0**	**20.9**

Message signing costs one Ed25519 sign and verify per batch (tens of microseconds). Reason codes take 2 to 4 ms and run only for batches that contain a held or frozen payment. Target: 50 ms at p99.

Decision rule (plain code on calibrated probabilities)

Freeze	p ≥ threshold giving 95% precision on the tune split; receiving bank asked to freeze
Hold	expected laundered dollars p × amount > $100 review cost
Hold at next hop	money came from a flagged payment within 72 h (hub accounts exempt)
Release	otherwise

Latency source: results/HI-Small/latency_inprocess_x86_64.json (run at 100×)

Innovation without exposure

Lifecycle

1. **Enroll:** the operator gives each bank a one-time token out of band; the bank registers its box key and verify key. The token is spent; later key changes need a request signed by the current key.
2. **Attest:** the enclave publishes keys, SHA-256 code measurement, model-file digests and Secure Execution status, signed. Each bank checks it against the expected measurement before sending anything. Production policy: hardware-bound only.
3. **Send:** envelope {from, to, nonce, issued_at, body}, signed by the sender, sealed to the recipient.
4. **Receive:** open → signature vs *registered* key → addressed to me → fresh → nonce unseen → validate → ownership.
5. **Answer:** only to the owner, signed by the enclave; banks reject anything the enclave didn't sign.

Who learns what

Party	Learns
Paying bank	decision on its own payment + pattern-family reason codes
Receiving bank	freeze request naming its own account and the amount
Other banks	nothing about the payment
Host operator	nothing inside the enclave (IBM Secure Execution)
Network / bus	ciphertext and routing metadata only

Attacks, defences, tests (all automated, all passing)

Attack	Defence
Submit as another bank	Ed25519 vs registered key
Member submits another bank's payment	ownership check
Replay a captured message	nonce cache + 120 s window
Tamper with ciphertext	sealed box authentication
Forge a "release" decision	banks verify enclave signature
Forward an envelope to another party	signed recipient field
Overwrite a bank's key	one-time token; signed rotation
Drain a bank's freeze notices	signed notice requests
Crash the enclave with junk	validation; refusals counted, never fatal
Fake or modified enclave	attestation: code + model digests
Raw accounts in features, logs, models	HMAC first; tests scan for raw strings

Production hardening on top: TLS/mTLS on the API and per-member ACLs on IBM Event Streams. Full threat model in SECURITY.md.

How FreezeFrame runs on LinuxONE

Item	Specification
Portability	Pure Python on Snap ML, which publishes s390x wheels; models are stored as portable XGBoost JSON and re-imported on the target, so a model trained anywhere scores on LinuxONE
Platform	IBM LinuxONE (s390x); IBM z16 / z17 or LinuxONE 4 and later for the Integrated Accelerator for AI
Datathon / pilot	LinuxONE Community Cloud VM, RHEL 9 or Ubuntu 22.04, s390x
Production enclave	IBM Secure Execution guest under IBM Confidential Computing Container Runtime (Hyper Protect Virtual Servers)
Python	3.10–3.12 (Snap ML 1.17.2 publishes s390x wheels)
Key libraries	snapml 1.17.2, xgboost <3 (or train on x86 and copy the portable JSON models), pynacl, fastapi, uvicorn
Sizing, HI-Small (5M payments)	8 vCPU, 16 GB RAM, 20 GB disk (the full experiment ran on an 8 GB laptop; enclave priming ~70–90 s per tier for 4M edges)
Sizing, HI-Medium (32M)	features ≈ 10.6 GB as float32: 32 GB RAM, or memory-mapped features (Roadmap)
Streaming bus	IBM Event Streams, or single-node Kafka (KRaft) on the VM; `payments.in` one partition; per-member topics with ACLs
Ports	8443 enclave HTTP API, 8000 demo dashboard (SSH tunnel), 9092 Kafka
Container	`deploy/Dockerfile` (python:3.11-slim, libgomp1); data, artifacts, results mounted as volumes, never baked in

Hardware checks (reported automatically by the app)

```
grep -o -w nnpa /proc/cpuinfo | head -1   # AI accelerator exposed?
cat /sys/firmware/uv/prot_virt_guest       # 1 = Secure Execution guest
```

The benchmark records `scoring_tree_format` (`zdnn_tensors` = AI accelerator, `compress_trees` = CPU); `/v1/attestation` and the dashboard badge show `secure-execution` or `simulated`.

Run on the VM

```
pip install -e '.[dev,xgb]' && python -m pytest
./scripts/quick_demo.sh                       # synthetic, minutes
./scripts/run_experiments.sh HI-Small         # real data, ~1 h
FF_TREE_FORMAT=zdnn_tensors python -m bench.latency --speeds 1 100 1000
FF_TREE_FORMAT=compress_trees python -m bench.latency --speeds 1 100
1000
```

Secure Execution contract (Hyper Protect)

- **workload:** signed s390x image of `deploy/Dockerfile`, port 8443
- **env** (encrypted to the HPVS key): pseudonym-key seed, enrollment-token hashes
- At boot the enclave generates its keys and puts `keys_hash` into the hardware attestation record; banks verify image measurement + keys hash (`allow_simulated=False`)

Step-by-step runbook: deploy/linuxone.md. Benchmarks write results/*/latency_*_s390x.json on LinuxONE.

5 | WHAT IS BUILT

A working system, end to end

Component	Where
Ingest, pseudonymisation, consortium split, alone vs together experiment	freezeframe/experiment.py, results/HI-Small/
Streaming graph features identical to training, no look-ahead	freezeframe/features.py, tests/test_features.py
Hot path and warm tier, hold at next hop, live model swap	freezeframe/enclave/, bench/latency.py
Signed and sealed transport, replay guard, enrollment, key rotation, attestation document	freezeframe/enclave/crypto.py, identity.py, attest.py
Reason codes that never reveal another bank's data	freezeframe/explain.py
Live dashboard with chain replay and attack panel	demo/
Kafka / IBM Event Streams transport and HTTP API	freezeframe/enclave/stream.py, service.py
Container image and LinuxONE runbook	deploy/

Next milestones

- Accelerator vs CPU latency on z16/z17 at several batch sizes
- Keys bound into the hardware Secure Execution attestation record
- Scale run on HI-Medium (32M payments) and IBM Event Streams with per-member ACLs

31 automated tests cover features, the enclave boundary, security and transport. Data: IBM AMLworld synthetic transactions (Altman et al., 2023); no real customer data is used.